Vue Directives Cheatsheet

Directives are special attributes prefixed with `v-` that tell Vue to do something to a DOM element. This is every built-in directive, its shorthand, and the modifiers you'll actually use.



Rendering

These control whether and how elements show up in the DOM.

```
<!-- v-if / v-else-if / v-else: adds or removes elements from the DOM entirely -->
<p v-if="status === 'loading'">Loading...</p>
<p v-else-if="status === 'error'">Something went wrong</p>
<p v-else>{{ data }}</p>

<!-- v-show: keeps the element in the DOM, toggles CSS display property -->
<p v-show="isVisible">I'm still in the DOM when hidden</p>

<!-- v-for: renders a list. Always provide a unique :key -->
<li v-for="item in items" :key="item.id">{{ item.name }}</li>
<li v-for="(item, index) in items" :key="item.id">{{ index }}: {{ item.name }}</li>
<li v-for="(value, key) in myObject" :key="key">{{ key }}: {{ value }}</li>

<!-- v-for with a range (starts at 1, not 0) -->
<span v-for="n in 5" :key="n">{{ n }}</span>
```

- **v-if vs v-show:** Use `v-if` when the condition rarely changes (avoids rendering cost). Use `v-show` when it toggles often (avoids mount/unmount cost).
- **v-if + v-for:** Never use them on the same element. Wrap `v-for` in a `<template>` and put `v-if` on a parent or child instead.

Binding

`v-bind` connects element attributes to reactive data. The `:` shorthand is what you'll use 99% of the time.

```
<!-- Full syntax vs shorthand -->
<img v-bind:src="imageUrl" />
<img :src="imageUrl" />

<!-- Spread multiple attributes from an object -->
<input v-bind="{ id: inputId, type: 'text', placeholder: hint }" />

<!-- Dynamic attribute name (computed at runtime) -->
<button :[attrName]="value">Click</button>

<!-- Class binding: object syntax toggles classes by truthiness -->
<div :class="{ active: isActive, 'text-bold': isBold }"></div>
<!-- Class binding: array syntax for combining static and dynamic -->
<div :class="[baseClass, isActive ? 'active' : '']"></div>

<!-- Style binding: camelCase keys map to CSS properties -->
<div :style="{ color: textColor, fontSize: size + 'px' }"></div>
```

Events

`v-on` listens for DOM events. The @ shorthand is the standard way to write it.

```
<!-- Full syntax vs shorthand -->
<button v-on:click="handleClick">Click</button>
<button @click="handleClick">Click</button>

<!-- Inline expression (fine for simple one-liners) -->
<button @click="count++">+1</button>

<!-- Passing the native event object explicitly -->
<button @click="handleClick($event)">Click</button>

<!-- Dynamic event name (useful for generic wrappers) -->
<button @[eventName]="handler">Click</button>
```

- **Event modifiers:** save you from calling `e.preventDefault()` or `e.stopPropagation()` manually:

```
<form @submit.prevent="onSubmit">          <!-- calls preventDefault() -->
<button @click.stop="onClick">             <!-- calls stopPropagation() -->
<div @click.self="onDiv">                  <!-- fires only if click target is the div itself -->
<button @click.once="onFirst">             <!-- auto-removes listener after first fire -->
<div @scroll.passive="onScroll">           <!-- hint to browser: won't call preventDefault -->
<a @click.stop.prevent="onLink">           <!-- modifiers can be chained -->
```

- **Key modifiers:** let you listen for specific keyboard keys:

```
<input @keyup.enter="submit" />            <!-- Enter key -->
<input @keyup.tab="nextField" />
<input @keyup.esc="cancel" />
<input @keyup.space="toggle" />
<input @keyup.ctrl.s="save" />             <!-- combo: Ctrl + S -->
<input @keyup.shift.enter="newLine" />     <!-- combo: Shift + Enter -->
<button @click.ctrl="selectMultiple" />    <!-- system modifier on click -->
<input @keyup.exact="onKeyUp" />           <!-- fires ONLY when no system modifiers held -->
```


Brought to you by certificates.dev

Learn more at certificates.dev/vuejs

Creators of the Official Vue.js Certification

- **Mouse modifiers:** distinguish between mouse buttons:

```
<div @click.left="onLeft" />              <!-- left click
(default) -->
<div @click.right="onRight" />            <!-- right click -->
<div @click.middle="onMiddle" />          <!-- middle/wheel
click -->
```

Custom Directives

When you need direct DOM access that doesn't fit into a component or composable, custom directives are the escape hatch.

```
// Inline directive in a component (prefix with v for auto-
registration)
const vFocus: Directive = {
  mounted(el: HTMLElement) {
    el.focus();
  },
};

// Usage: <input v-focus />
```

```
// Global registration in main.ts (or a Nuxt plugin)
app.directive('click-outside', {
  mounted(el, binding) {
    // binding.value is whatever you pass: v-click-
outside="closeMenu"
    el._handler = (e: Event) => {
      if (!el.contains(e.target as Node)) {
        binding.value(e);
      }
    };
    document.addEventListener('click', el._handler);
  },
  unmounted(el) {
    // Always clean up global listeners
    document.removeEventListener('click', el._handler);
  },
});

// Usage: <div v-click-outside="closeMenu">...</div>
```

- **Directive hooks:** `created`, `beforeMount`, `mounted`, `beforeUpdate`, `updated`, `beforeUnmount`, `unmounted`.

Most custom directives only need `mounted` and `unmounted`.



Other Built-in Directives

These are less common but useful in specific situations.

```
<!-- v-text: sets element's textContent. Usually {{ }}
interpolation is cleaner -->
<span v-text="message" />

<!-- v-html: sets innerHTML. Only use with trusted content (XSS
risk) -->
<div v-html="rawHtml" />

<!-- v-pre: skips compilation. Good for displaying raw template
syntax -->
<span v-pre>{{ this will not compile }}</span>

<!-- v-once: renders once, then treats element as static. Good
for unchanging content -->
<h1 v-once>{{ title }}</h1>

<!-- v-memo: caches the subtree until listed deps change. Use
for expensive v-for lists -->
<li v-for="item in list" :key="item.id" v-memo="[item.name]">
  {{ item.name }}
</li>

<!-- v-cloak: hides element until Vue mounts. Prevents flash of
raw {{ }} on slow loads -->
<div v-cloak>{{ message }}</div>
<!-- Pair with CSS: [v-cloak] { display: none; } -->
```

v-model

Two-way binding for form elements. Under the hood it's a `:value` + `@input` combo.

```
<!-- Text inputs -->
<input v-model="name" />
<textarea v-model="bio" />

<!-- Select dropdown -->
<select v-model="selected">
  <option value="a">A</option>
  <option value="b">B</option>
</select>

<!-- Single checkbox: binds to a boolean -->
<input type="checkbox" v-model="agreed" />

<!-- Multiple checkboxes with same v-model: binds to an array
-->
<input type="checkbox" v-model="selectedTags" value="vue" />
<input type="checkbox" v-model="selectedTags" value="nuxt" />
```

- **v-model modifiers:** change when or how the value syncs:

```
<input v-model.lazy="query" />     <!-- syncs on "change" event
instead of "input" -->
<input v-model.number="age" />     <!-- auto-casts value to a
number -->
<input v-model.trim="name" />      <!-- strips leading/trailing
whitespace -->
<input v-model.lazy.trim="bio" />  <!-- modifiers can be
chained -->
```

Brought to you by certificates.dev

Learn more at certificates.dev/vuejs

Creators of the Official Vue.js Certification